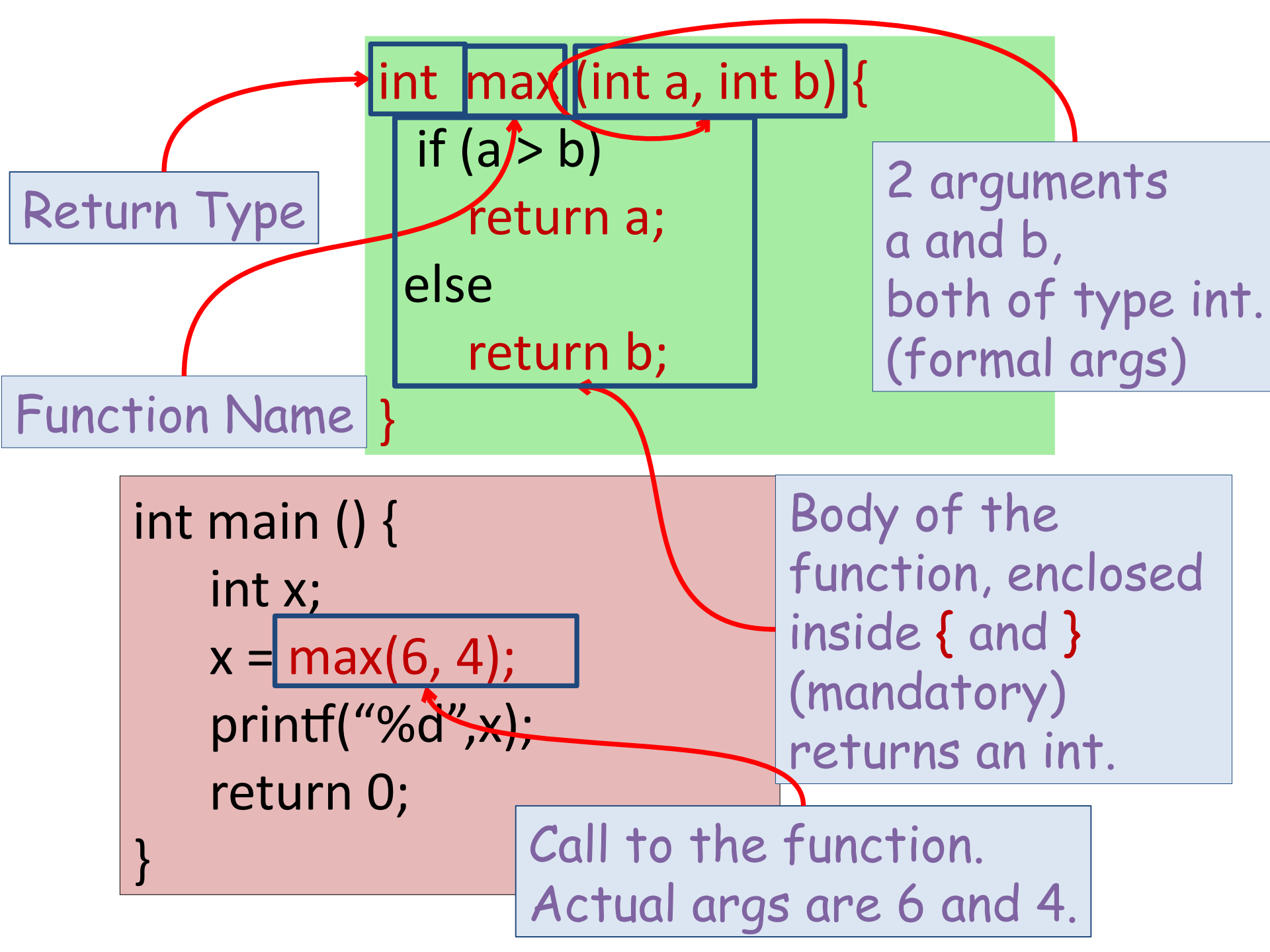
More about Functions

CSL100

September 2, 2024

This Class

- Functions
  - Scope of a variable
  - Execution trace
  - Parameter passing


int max (int a, int b) {
if (a > b)
return a;
else
return b;
}
Return Type
Function Name
2 arguments
a and b,
both of type int.
(formal args)
int main () {
int x;
x = max(6, 4);
printf("%d",x);
return 0;
}
Body of the
function, enclosed
inside { and }
(mandatory)
returns an int.
Call to the function.
Actual args are 6 and 4.

Example Program

- Problem description: write a program to accept two positive integers and return all the prime numbers lying between them
- Solution approach
  - Use main function for I/O
  - Use a loop to run through all numbers in the range
  - Use a custom function to check for primality of each number
  - Primality test:
    - Run a loop over possible divisors
    - Assign a flag value to 1 if the number is divisible by a smaller number
    - Return ‘prime’ if flag stays zero at the end of the loop

Prototype

Function call

Definition

```c
#include <stdio.h>

int checkPrimeNumber(int n);

int main() {
        int n1, n2, i, flag;
        printf("Enter two positive integers: ");
        scanf("%d %d", &n1, &n2);
        printf("Prime numbers between %d and %d are: ", n1, n2);

        for(i=n1+1; i<n2; ++i) {
        // i is a prime number, flag will be equal to 1
                flag = checkPrimeNumber(i);
                if(flag == 1)
                        printf("%d ",i);
         }
        return 0;
}

// user-defined function to check prime number
int checkPrimeNumber(int n) {
         int j, flag = 1;
        for(j=2; j <= n/2; ++j) {
                if (n%j == 0) {
                        flag =0;
                        break;
                }
        }
        return flag;
}
```

Avoiding Common Errors

- Declare functions before use
- Argument list of a function:
  - Provide the required number of arguments
  - Check that each function argument has the correct type (or that conversion to the correct type will lose no information)

Avoiding Common Errors

- Return statement
  - value must match the return type
  - return statement must be encountered during execution for a function having non void return type
- Also be careful in using functions that are undefined on some values.
  - is defined only for
  - function in C : double asin(double x)
    - pronounced a-sine or arc-sine

Scope of a Name

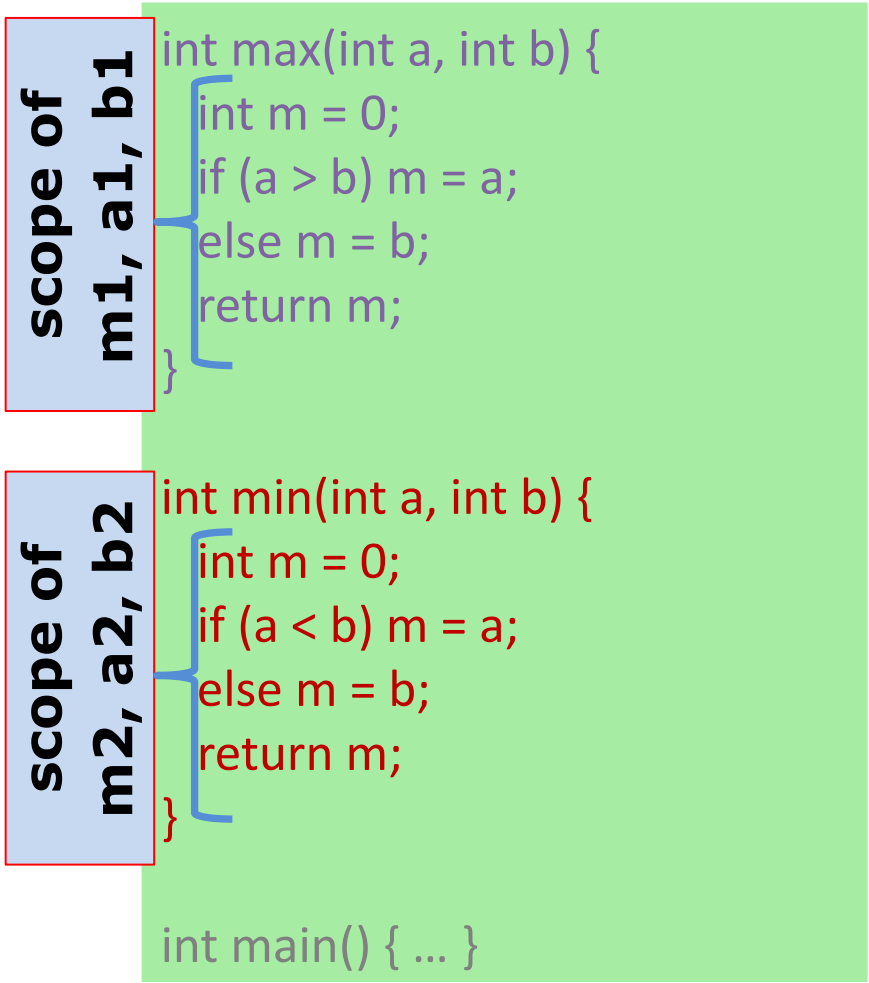
- Functions allow us to divide a program into smaller parts
  - each part does a well defined task
- There are other ways to *partition a program*
  - *Statement blocks, Files*
- Scope of a name is the part of the program in which the name can be used

Scope of a Name

- Two variables can have the same name only if they are declared in separate scopes
- A variable can not be used outside its scope
- C program has
  - function/**block** scope
  - **file** scope
  - **global**/external scope

Scope Rules: Functions

- The scope of the variables present in the argument list of a function's definition is the body of that function
- The scope of any variable declared within a function is the body of the function



Scope Rules : Blocks

- For an identifier declared inside a block
  - Scope begins at the end of declaration
  - Scope ends at the end of the block

```
int main() {
   int m = 5;
   {
       float m = 6.5; // shadows
       printf("%f", m);  // prints 6.5
   }
   printf("%d" , m); // prints 5
}
```

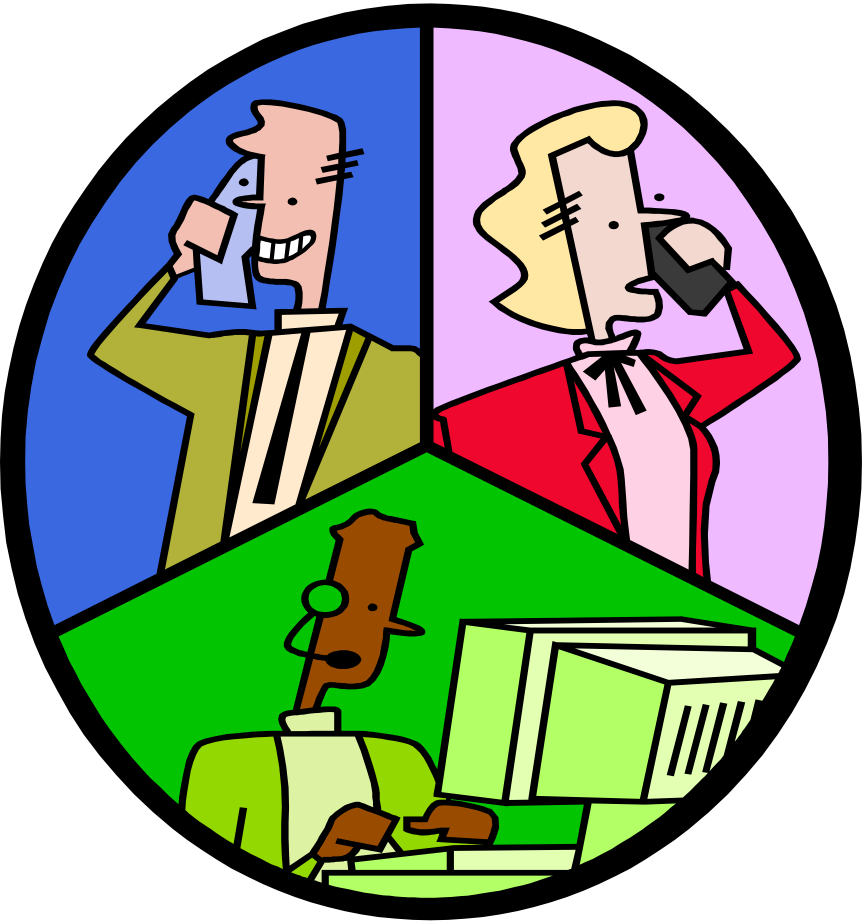
Scope of a Name

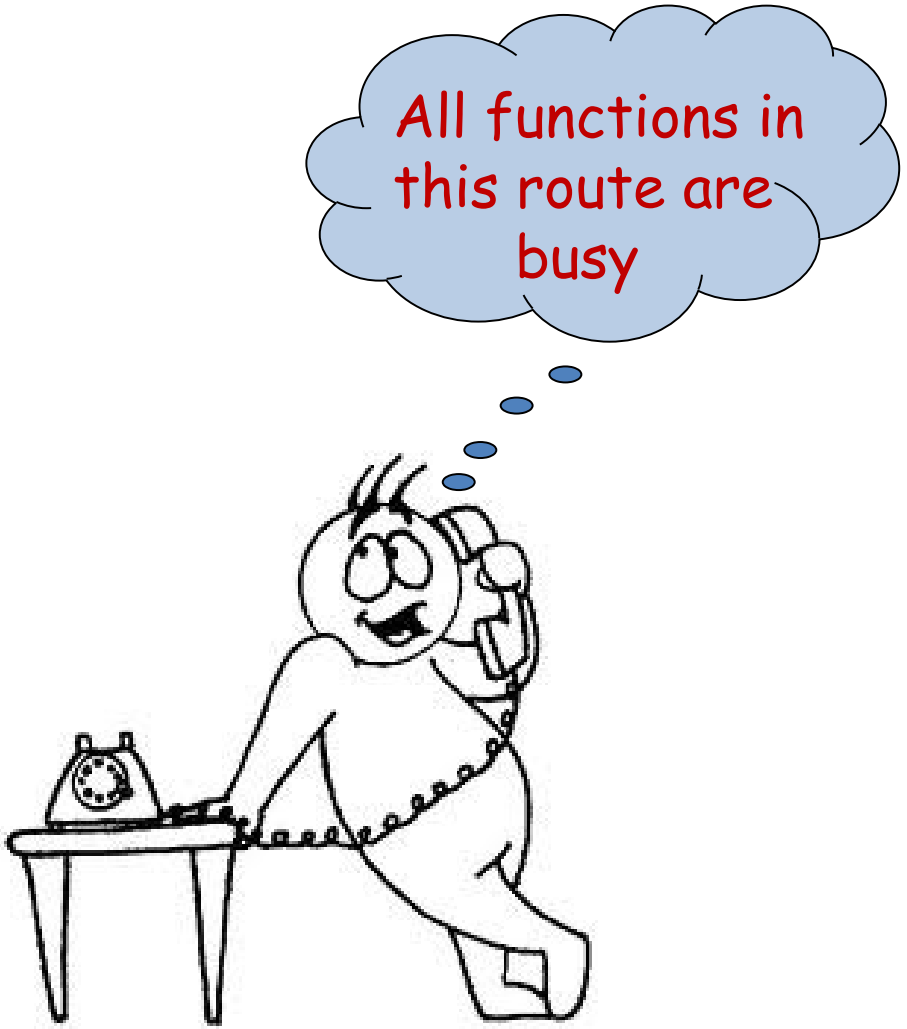
- Scopes can be nested
  - A name in inner scope "*shadows*" the same name, if present in outer scope

```
int main() {
   int m = 5;
   {
        float m = 6.5; // shadows
        printf("%f", m);  // prints 6.5
   }
   printf("%d" , m); // prints 5
}
```



Execution of a Function: Steps





```
1   #include <stdio.h>
2   int max(int a, int b) {
3     if (a > b)
4         return a;
5     else
6         return b;
7   }

8   int main () {
9         int x;      10a
10        x =         max(6, 4);
11        printf("%d",x);
12        return 0;
13  }
```

- Steps when a function is called: max(6,4) in step 10a.
- Allocate space for (i) return value, (ii) store return address and (iii) pass parameters.

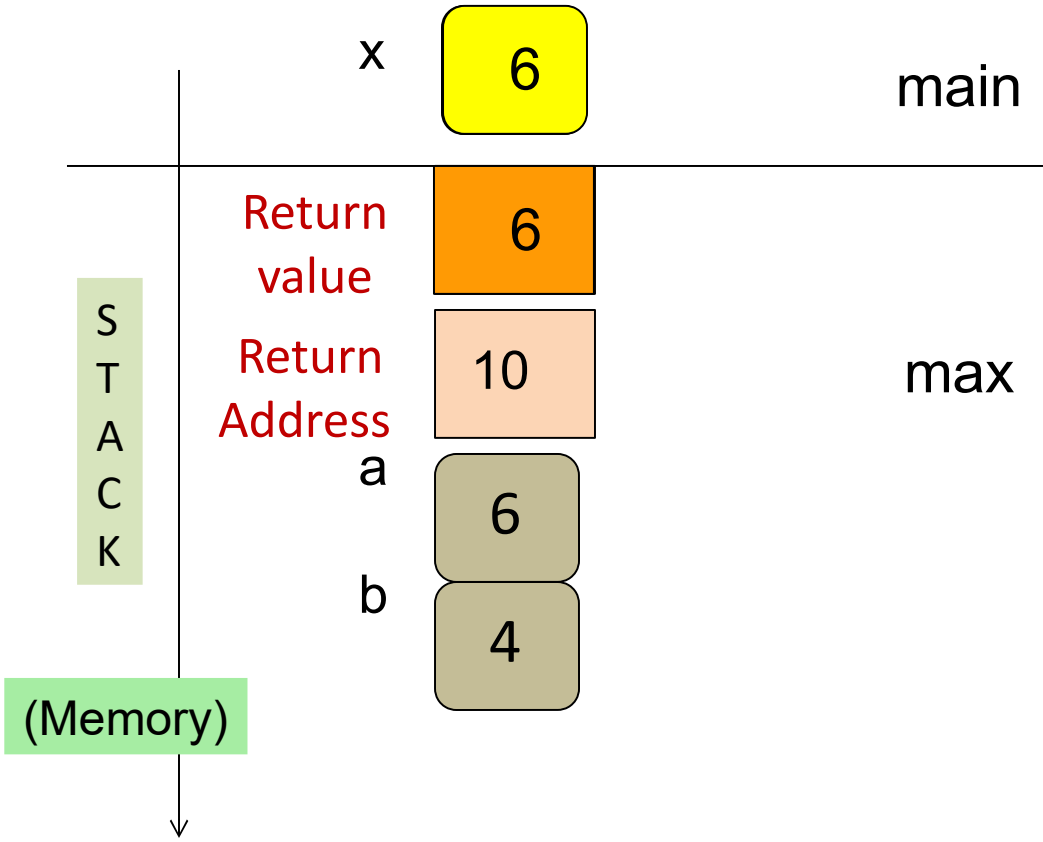
1. Create a box informally called ``Return value'' of same type as the return type of function.
2. Create a box and store the location of the next instruction in the calling function (main)—return address. Here it is 10 (why not 11?). Execution resumes from here once function terminates.
3. Parameter Passing- Create boxes for each formal parameter: a, b here. Initialize them using actual parameters, 6 and 4.

```
1   #include <stdio.h>
2   int max(int a, int b) {
3     if (a > b)
4       return a;
5     else
6         return b;
7   }

8   int main() {
9     int x = -1;
10    x = max(6, 4);
11    printf("%d",x);
12        return 0;
13  }
```

Calling max(6,4):

1. Allocate space for return value.
2. Store return address (10).
3. Pass parameters.



After completing max(), execution in main() will re-start from address 10.

Stack

- We referred to stack.
- A stack is just a part of the memory of the program that grows in one direction only.
- The memory (boxes) of all variables defined as actual parameters or local variables reside on the stack.
- The stack grows as functions call functions and shrinks as functions terminate.



Evaluating Expressions

- Associativity and Precedence precisely define what an expression means, e.g.,

  a * b – c / d

  is same as:

  (a * b) – (c / d)

- But how exactly are expressions evaluated?

Evaluating Expression (a*b) – (c/d)

- Computers evaluate one operator at a time
- Above expr may be evaluated as

```
t1 = c/d;
t2 = a*b;
t3 = t2 – t1;
```

  - t1, t2, t3 are temporary variables, created by the compiler (not by programmer)
  - They are temporary, meaning, their lifetime is only until use

Evaluating Expression (a*b) – (c/d)

- The order of evaluation of arguments for most C operators is not defined
  - Exceptions are && and ||
  - Remember: short circuit evaluation
- Above expr may also be evaluated as

```
t1 = a*b;
t2 = c/d;
t3 = t1 – t2;
```

  - Do not depend on order of evaluation of args, except for && and ||
  - Same holds true for function arguments

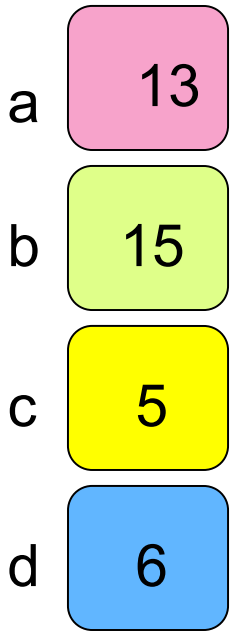
Evaluating Expression a++ * ++a

- a = 10
- a++ * ++a = 120 (left to right)
- a++ * ++a = 121 (right to left)

AVOID

```
int main () {
    int a =4, b = 2, c = 5, d = 6;
    a = a + b *c-d/a;
    b = b-(c-d)*a;
    printf("%d %d", a, b);
    return 0; }
```

State of the program just prior to printf



Output

13 15

Since each function call is also an expression, statements having mix of function calls and operators are also evaluated in a similar fashion.

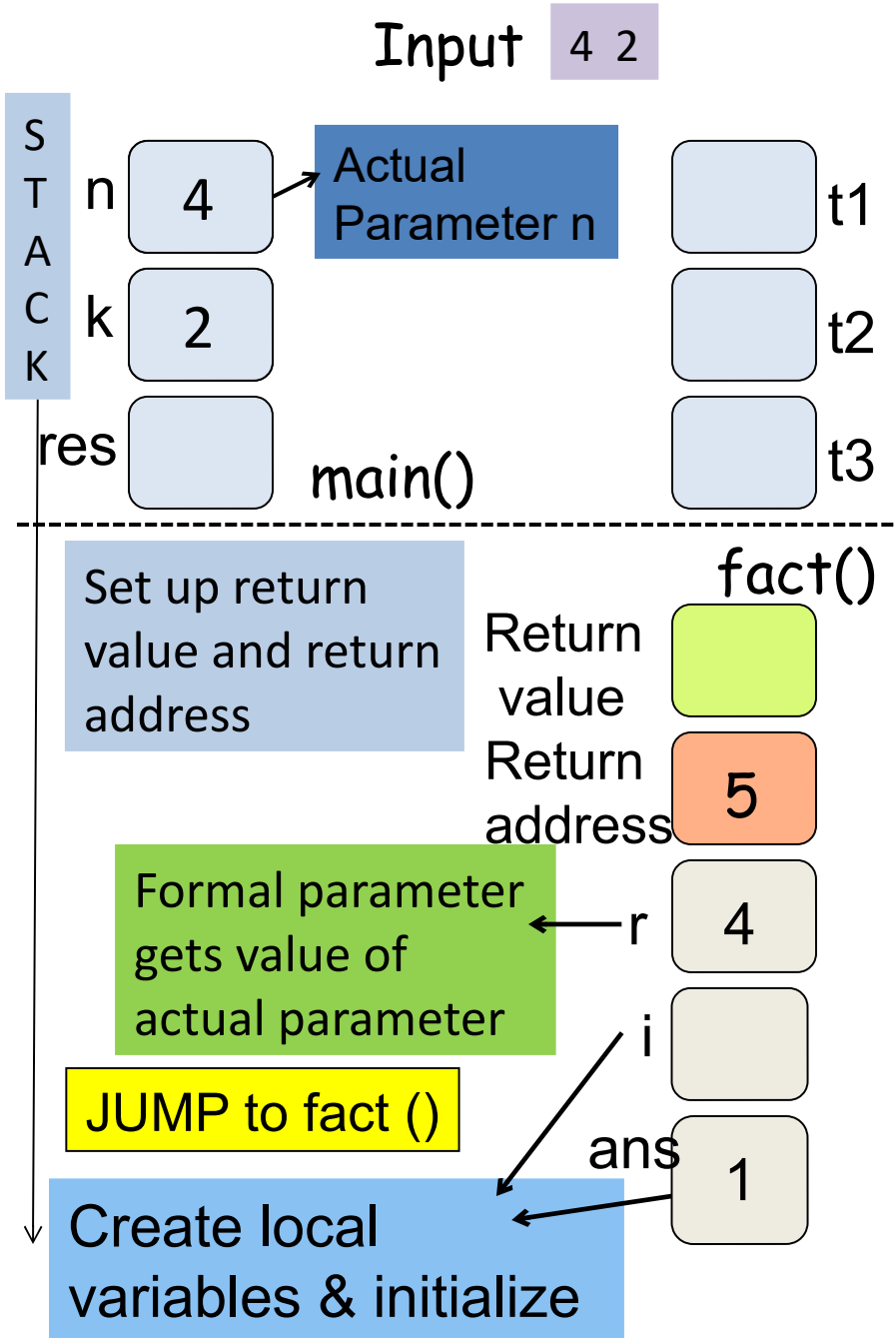
```
# include <stdio.h>
int fact(int r) {   /* calc. r! */
      int i;
      int  ans=1;
      for (i=0; i < r; i=i+1) {
            ans = ans *(i+1);
      }
      return ans;
}
```

```
int main () {
      int n, k;
      int res;
      scanf("%d%d",&n,&k);
      res = (fact(n)/ fact(k))/fact(n-k);
      printf("%d choose %d is",n,k);
      printf("%d\n",res);
      return 0;
}
```

- Define a factorial function.
- Use to calculate ${}^{n}C_{k}$
- Let us trace the execution of main().
- Add temporary variables for expressions and intermediate expressions in main for clarity.

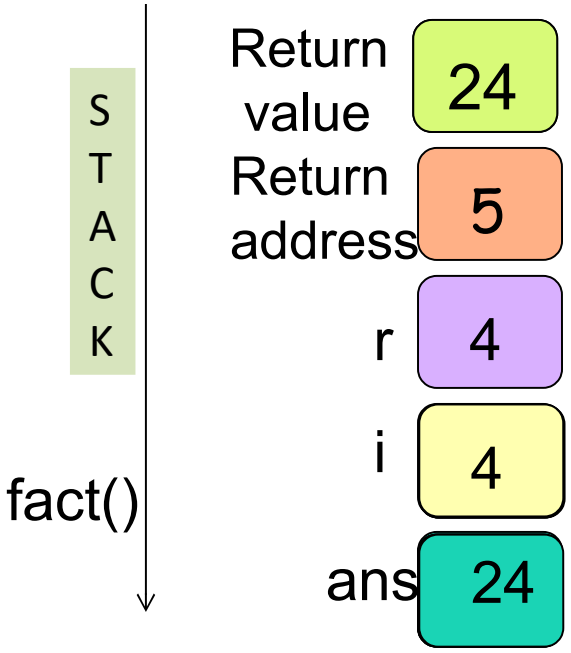
```
int fact(int r) {   /* calc. r! */
    int i; int ans =1 ;
        /*  code here */
}
```

```
1  int main () {
2        int n, k;
3        int res;
4        scanf("%d%d",&n,&k);
     /* Adding temporary vars and
     Intermediate exprs. */
               int      t1, t2, t3;
5              t1 =     fact(n);
6              t2 =     fact(k):
7              t3 =     fact(n-k);
8              res =   (t1/t2)/t3;
         printf("%d choose %d is");
9        printf("%d\n", res);
         return 0;
     }
```



```
# include <stdio.h>
int fact(int r) {   /* calc. r! */
     int i;
     int ans=1  ;
     for ( i=0;   i < r;    i=i+1) {
       ans = ans *(i+1);
     }
     return ans;
}
```

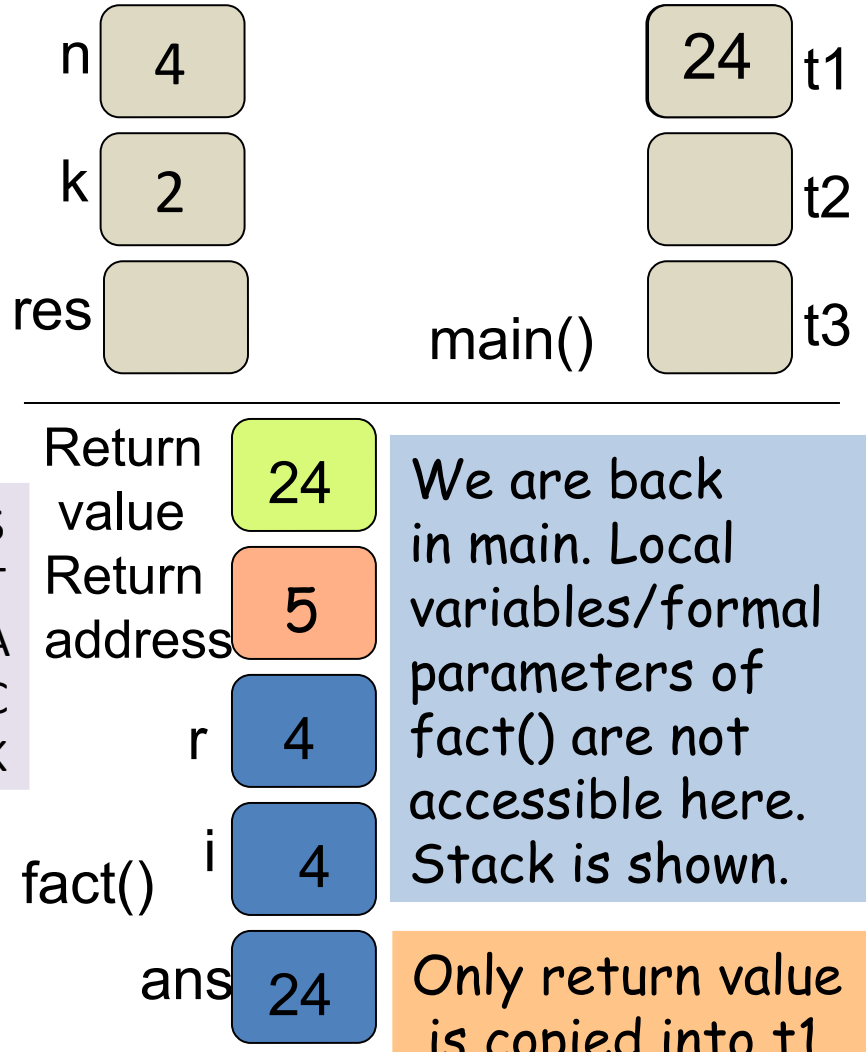
We have jumped to fact() and prepared the stack for the call. Parameters are passed, return addr is stored and local variables are initialized. Now we are ready to execute.



Assign to return value

Now jump to return address

```
1  int main () {
2        int n, k;
3        int res;
4        scanf("%d%d",&n,&k);
     /* Adding temporary vars
      and Intermediate exprs. */
               int       t1, t2, t3;
5              t1 =      fact(n);
6              t2 =      fact(k);
7              t3 =      fact(n-k);
8              res =   (t1/t2)/t3;
       printf("%d choose %d is");
9      printf("%d\n",res);
       return 0;
   }
```



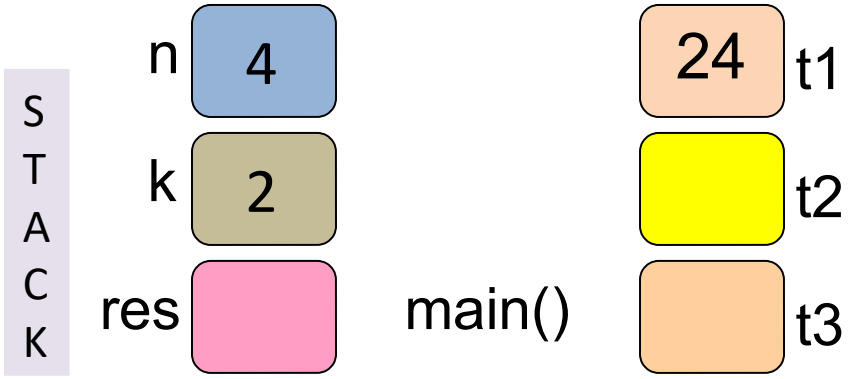
We are back in main. Local variables/formal parameters of fact() are not accessible here. Stack is shown.

Only return value is copied into t1.

Control jumps to statement 5, since that was the return address.

After copying return value, the stack for fact() is wiped clean.

```
1  int main () {
2        int n, k;
3        int res;
4        scanf("%d%d",&n,&k);
     /* Adding temporary vars
      and Intermediate exprs. */
               int       t1, t2, t3;
5              t1 =      fact(n);
6              t2 =      fact(k);
7              t3 =      fact(n-k);
8              res =   (t1/t2)/t3;
       printf("%d choose %d is");
9      printf("%d\n",res);
       return 0;
   }
```



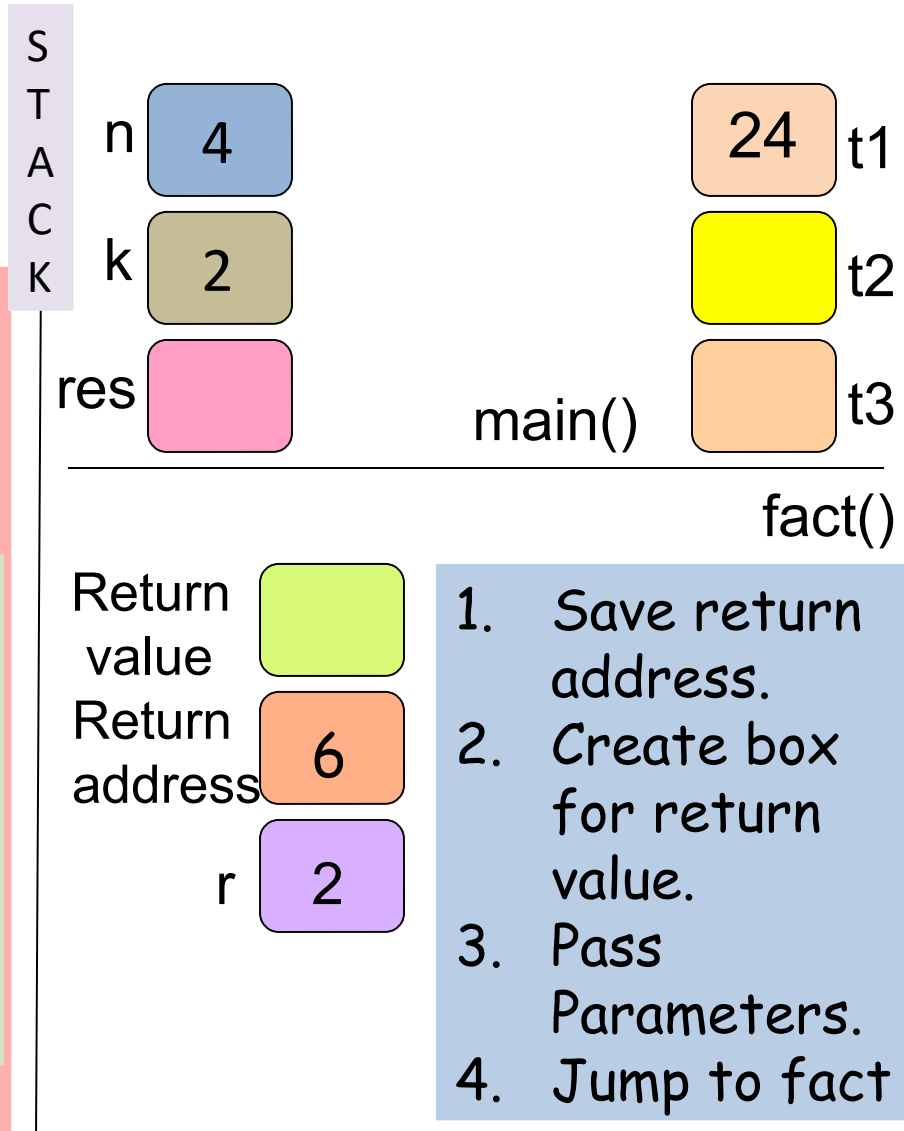
The next statement is another function call: fact(k). Prepare stack for new call.

1. Save return address.
2. Create box for return value.
3. Pass Parameters: Create boxes corresponding to formal parameters. Copy values from actual parameters.
4. Jump to called function.
5. Create/initialize local variables.

After copying return value, the stack for fact() is wiped clean.

```
int fact(int r) {   /* calc. r! */
    int i; int ans =1 ;
        /*  code here */
}
```

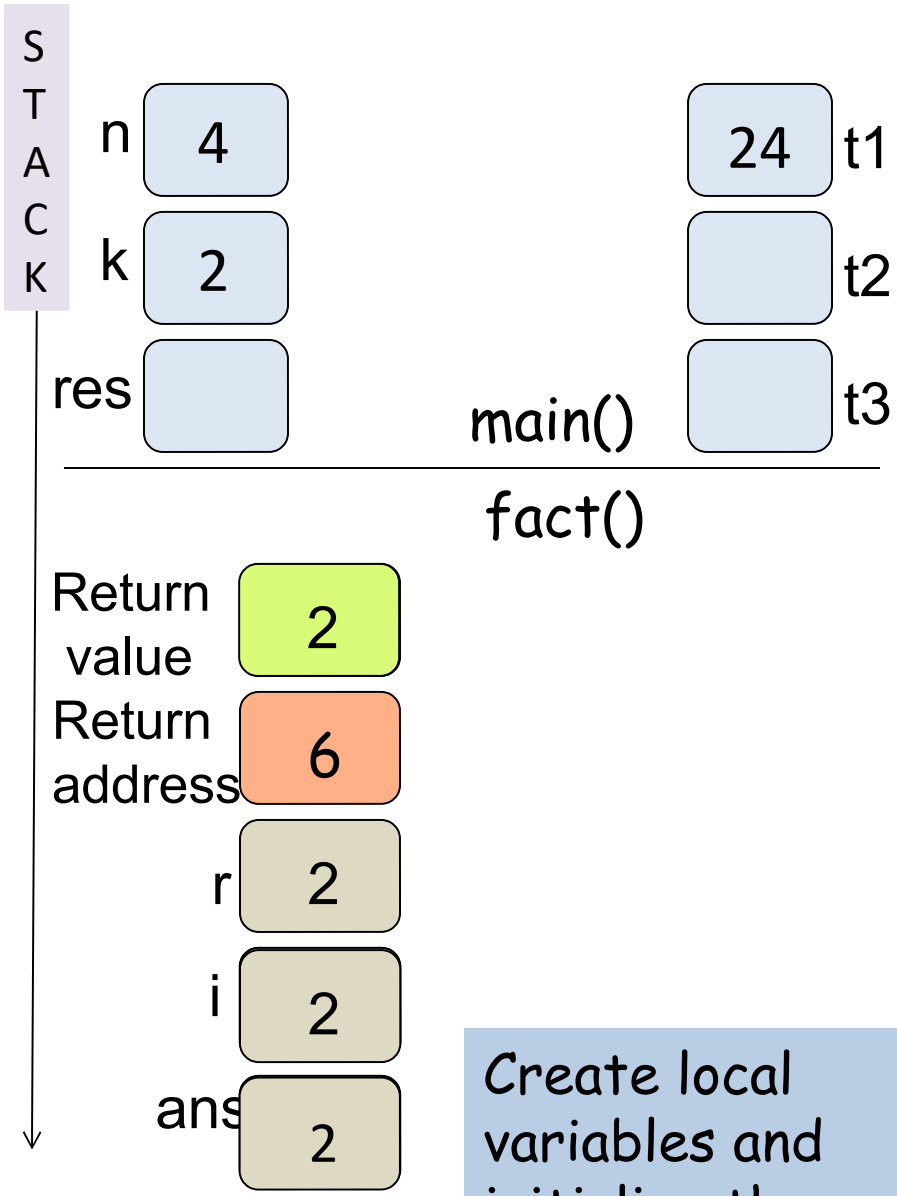
```
1  int main () {
2      int n, k;
3      int res;
4      scanf("%d%d",&n,&k);
       /* Adding temporary vars and
       Intermediate exprs. */
               int     t1, t2, t3;
5              t1 =    fact(n);
6              t2 =    fact(k);
7              t3 =    fact(n-k);
8              res =   (t1/t2)/t3;
       printf("%d choose %d is");
9      printf("%d\n",res);
       return 0;
   }
```



1. Save return address.
2. Create box for return value.
3. Pass Parameters.
4. Jump to fact

```
# include <stdio.h>
int fact(int r) {   /* calc. r! */
      int i;
      int  ans=1;

      for (i=0;  i < r; i=i+1) {
            ans = ans *(i+1);
      }
      return ans;
}
```

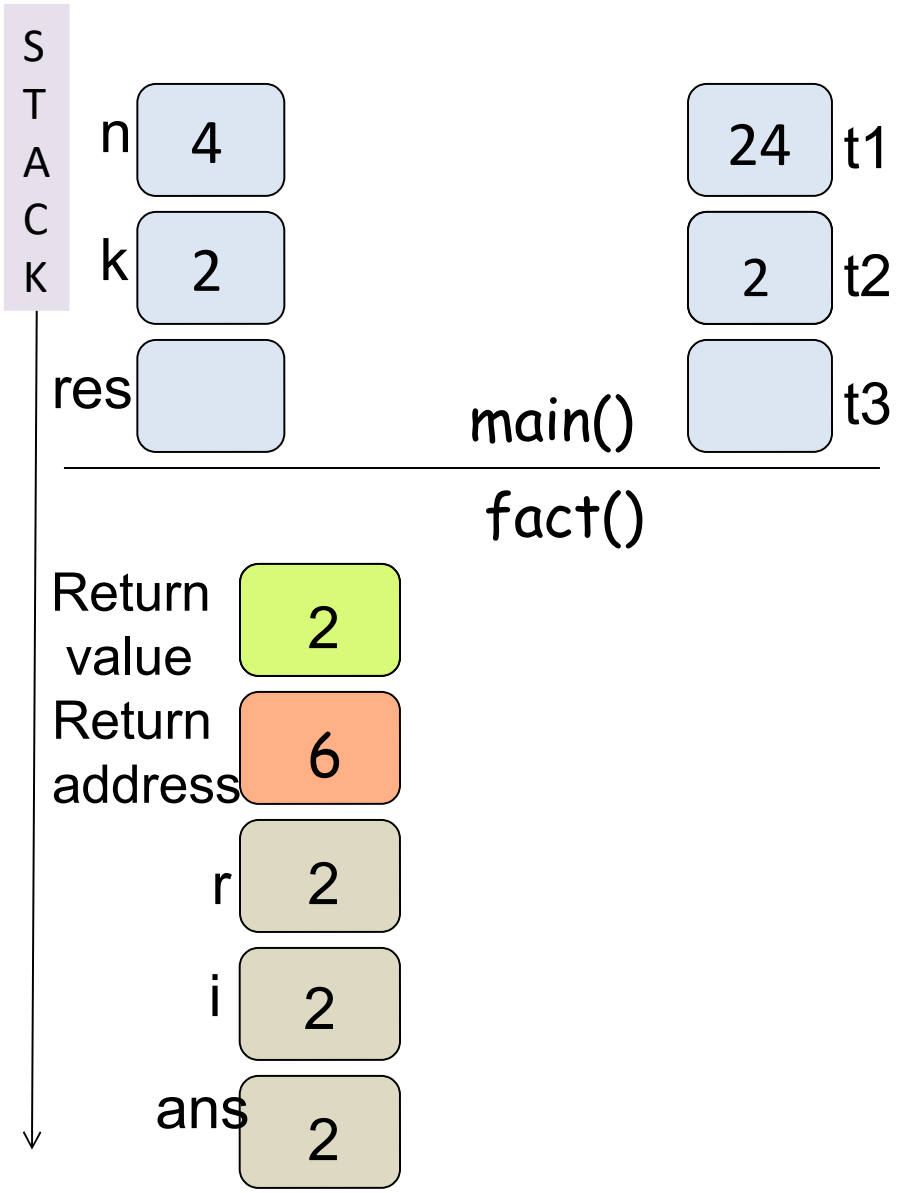


Assign value of ans to box for return value.

Create local variables and initialize them

Now jump to return address

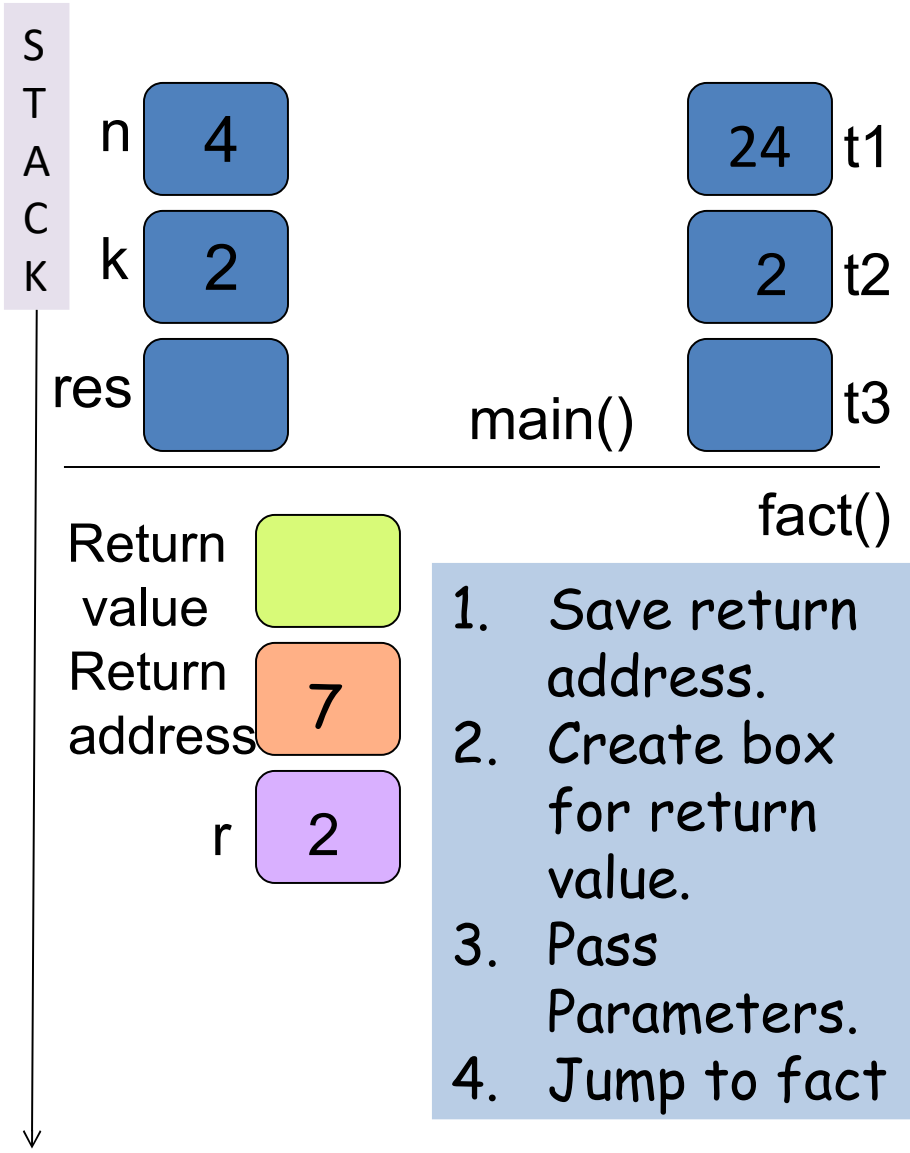
```
1  int main () {
2        int n, k;
3        int res;
4        scanf("%d%d",&n,&k);
     /* Adding temporary vars
     and Intermediate exprs. */
               int       t1, t2, t3;
5              t1 =      fact(n);
6              t2 =      fact(k):
7              t3 =      fact(n-k);
8              res =   (t1/t2)/t3;
         printf("%d choose %d is");
9        printf("%d\n",res);
         return 0;
   }
```



This is another function call. So we prepare stack. Earlier entries for fact() is erased.

```
int fact(int r) {   /* calc. r! */
    int i; int ans =1 ;
        /*  code here */
}
```

```
1  int main () {
2       int n, k;
3       int res;
4       scanf("%d%d",&n,&k);
        /* Adding temporary vars
         and Intermediate exprs. */
                int      t1, t2, t3;
5               t1 =     fact(n);
6               t2 =     fact(k):
7               t3 =     fact(n-k);
8               res =   (t1/t2)/t3;
        printf("%d choose %d is");
9       printf("%d\n",res);
        return 0; }
```

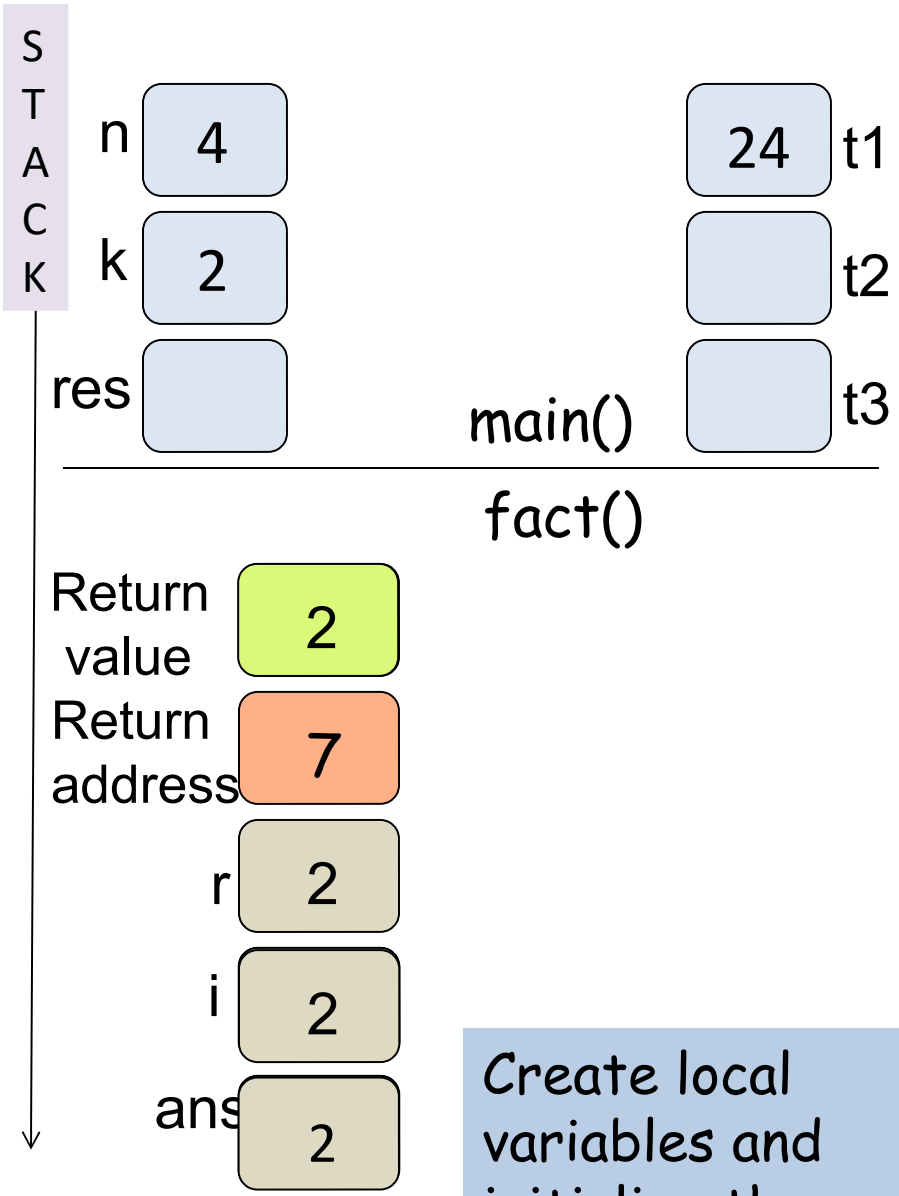


1. Save return address.
2. Create box for return value.
3. Pass Parameters.
4. Jump to fact

Previous stack for fact() has been erased.

```
# include <stdio.h>
int fact(int r) {   /* calc. r! */
      int i;
      int  ans=1;

      for (i=0;  i < r; i=i+1) {
            ans = ans *(i+1);
      }
      return ans;
}
```

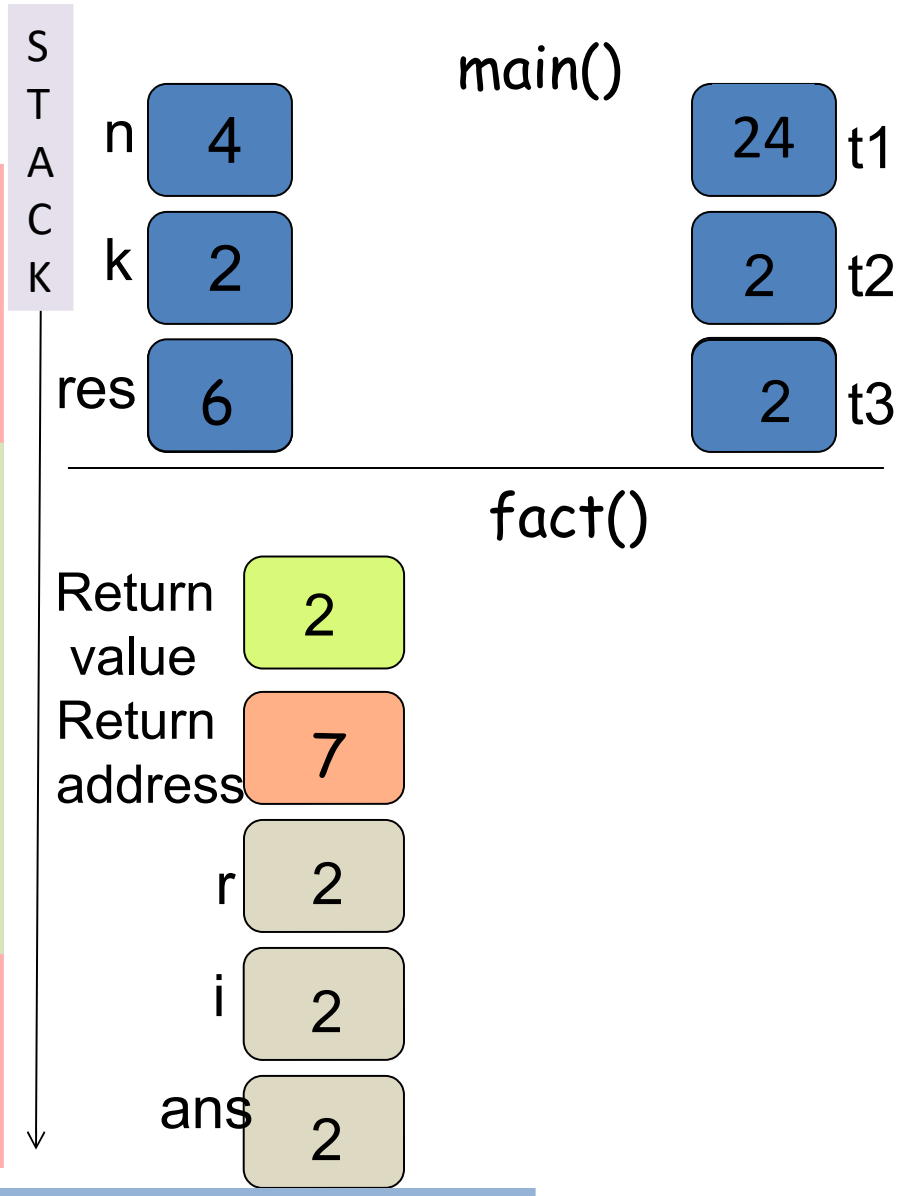


Assign value of ans to box for return value.

Create local variables and initialize them

Now jump to return address

```
1  int main () {
2        int n, k;
3        int res;
4        scanf("%d%d",&n,&k);
     /* Adding temporary vars
      and Intermediate exprs. */
               int      t1, t2, t3;
5              t1 =     fact(n);
6              t2 =     fact(k):
7              t3 =     fact(n-k);
8              res =   (t1/t2)/t3;
       printf("%d choose %d is ");
9      printf("%d\n",res);
       return 0; }
```



Stack for fact() is erased after return value is copied.

4 choose 2 is 6

Next Class

- Functions
  - Library Functions
  - Macros
  - More examples